Waffle 5.1 fast (high) — PO Yönetim Sistemi

CSE 311 & CSE 321 ortak proje raporu · 12 Kasım 2026 · **Ekip:** EmreSndkc · brkznqr · eminozkanca · Berkegn · **Repo:** https://github.com/EmreSndkc/waffle_v5.1

CSE 311 — 1 Punchline & Backlog

Punchline: Waffle 5.1, hocanın girdiği notlardan her öğrencinin 11 Program Outcome (PO) puanını otomatik hesaplayan ve bölüme akreditasyon için hazır bir PO profili veren Django web sistemidir.

Backlog: 16 özellik, Fibonacci story point ile puanlandı (M = temel, I = yenilikçi; toplam 64 puan). Backlog şu an bu tablodur; GitHub Projects board'una issue olarak taşınması S1'in ilk işidir.

#	Özellik	Tür	Puan	Durum
1	Giriş/çıkış, rol bazlı yönlendirme	M	3	S1
2	PO yönetimi + 11 PO seed verisi	M	2	**Bitti (S0)**
3	Ders açma, hoca atama, öğrenci kaydı	M	3	Yalnızca admin panelinde
4	LO + değerlendirme + yüzde girme ekranı	M	5	Yalnızca admin panelinde
5	LO→PO eşleme matrisi (1–5)	M	3	Yalnızca admin panelinde
6	Not girme ekranı	M	3	Yalnızca admin panelinde
7	PO hesaplama motoru + snapshot + otomatik yeniden hesaplama	M	5	**Bitti (S0)**
8	Öğrenci paneli: PO grafiği	M	3	S3
9	Excel/CSV ile toplu not yükleme	I	5	S3
10	Syllabus PDF'inden LO ve ağırlıkları çıkarma	I	8	S4
11	PDF performans raporu	I	5	S4
12	Bölüm başkanı paneli: PO × ders ısı haritası	I	5	S5
13	Risk uyarısı e-postası (PO eşiğin altında)	I	3	S5
14	"What-if" simülatörü	I	3	S5
15	Değişiklik geçmişi (audit log)	I	3	S5
16	REST API + Türkçe/İngilizce arayüz	I	5	Zaman kalırsa

CSE 311 — 2 Work & Velocity

Puanlama: Fibonacci story point (1, 2, 3, 5, 8); puan özelliğin zorluğunu gösterir. **Sprint süresi:** 1 hafta; başında planlama, sonunda review ve `develop` → `main` merge. **Tamamlanan sprint:** 1 (S0, 11 Kasım'da kapandı).

Velocity: S0 = **7 puan** (#2 ve #7). Yalnızca tamamen biten özellikler sayıldı; #3–#6 admin panelinden kullanılabiliyor ama role özel ekranları olmadığı için sayılmadı. Kalan 57 puan altı sprinte bölünüyor; gereken ortalama 9,5 puan/sprint.

Üye	S0'da yaptığı kod işi	Diyagram	Puanlı özellik
EmreSndkc	Django iskeleti, ortam değişkenli ayarlar (PostgreSQL/Redis/SMTP), README	Sistem mimarisi	altyapı (puansız)
brkznqr	Model düzeltmeleri: `Enrollment`, kısıtlar, index'ler, migration	ER diyagramı	#7'nin veri modeli
eminozkanca	Hesaplama servisi, otomatik yeniden hesaplama (signals), 15 test	Class diyagramı	#7 (5 puan)
Berkegn	`seed_demo` komutu, Django admin paneli	Rol akış şeması	#2 (2 puan)

Her iş `feature/<ad>` dalında yapıldı ve sahibinin adıyla commit'lendi; beş dal `develop`'a, `develop` `main`'e merge edildi. Testler (15) SQLite ve PostgreSQL 18'de geçiyor.

CSE 321 — 1 Introduction

Hesap zinciri üç basamaklıdır: değerlendirme notu → (yüzdelerle) Learning Outcome (LO) → (1–5 ağırlıkla) PO. Her basamakta aynı formül kullanılır: `puan = Σ(ağırlık × puan) ÷ Σ ağırlık`. Kabul testi hocanın örneğidir: Midterm 90, Project 80 için sistem LO1 = 86.0, LO2 = 82.9, PO1 = 85.0, PO2 = 82.9 üretir (birim testi ve demo verisindeki `elif` kullanıcısı).

Temel özellik	Durum
Veri modeli: kullanıcı/rol, ders, kayıt, LO, PO, değerlendirme, not, eşlemeler	Çalışıyor
PO hesaplama servisi ve ön hesaplanmış PO profili (`StudentPOSnapshot`)	Çalışıyor
Not, eşleme ya da ağırlık değişince otomatik yeniden hesaplama	Çalışıyor
Verisi olmayan PO için 0 yerine NULL ("veri yok")	Çalışıyor
Django admin paneli ve demo verisi (`python manage.py seed_demo`)	Çalışıyor
Rol bazlı ekranlar, öğrenci paneli, toplu not yükleme, raporlar	Sonraki sprintler

Rol	Girer	Görür
Sistem Yöneticisi	Kullanıcılar, roller	Her şey
Bölüm Başkanı	PO'lar, dersler, hoca ataması, öğrenci kaydı	Tüm dersler ve tüm öğrencilerin PO puanları
Öğretim Üyesi	LO'lar, değerlendirmeler, yüzdeler, LO→PO ağırlıkları, notlar	Dersler, notlar, PO puanları
Öğrenci	—	Kendi notları ve PO profili

Bugün yetkiler Django admin'de model düzeyindedir; "hoca yalnızca kendi dersini görür" kontrolü ve öğrenci paneli henüz yoktur.

CSE 321 — 2 System Design

Mimari (Şekil 1). Sistem tek bir Django uygulama sunucusudur: istek `config/` altındaki URL ve ayarlardan geçer, `outcomes/` uygulamasında modellere ve hesaplama servisine ulaşır. Dış bileşenler `config/settings.py` içinde ortam değişkeniyle seçilir; değişken yoksa yerel karşılığı kullanılır.

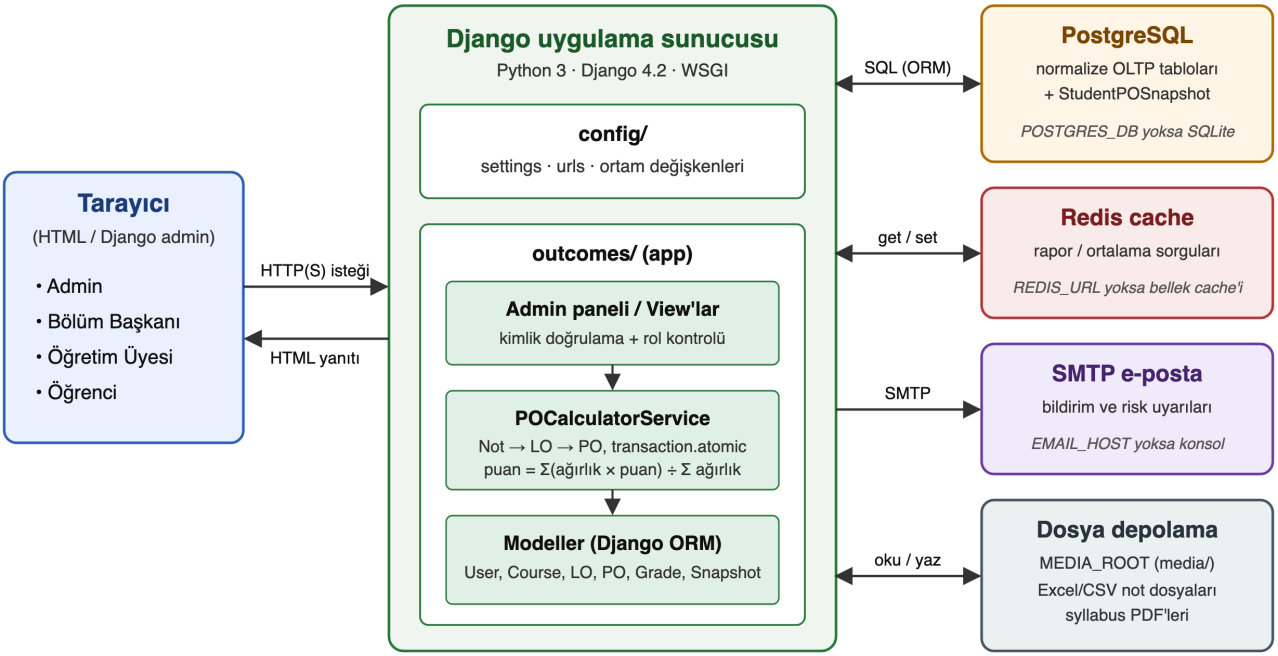
Bileşen	Görevi	Değişken	Yoksa	Bugünkü durum
PostgreSQL	Kalıcı veri	`POSTGRES_DB`	SQLite	Kullanılıyor; testler PostgreSQL 18'de geçiyor
Redis	Rapor sorgularının cache'i	`REDIS_URL`	Süreç içi bellek	Yapılandırıldı; cache'lenen sorgu henüz yok
SMTP	E-posta bildirimi	`EMAIL_HOST`	Konsol	Yapılandırıldı; e-posta gönderen özellik henüz yok
Dosya depolama	Excel/CSV, syllabus PDF	`MEDIA_ROOT`	`./media`	Yapılandırıldı; yükleme ekranı henüz yok

Sınıflar (Şekil 2). Kod üç parçadır: modeller (`models.py`), hesaplama servisi (`services.py`) ve tetikleyiciler (`signals.py`).

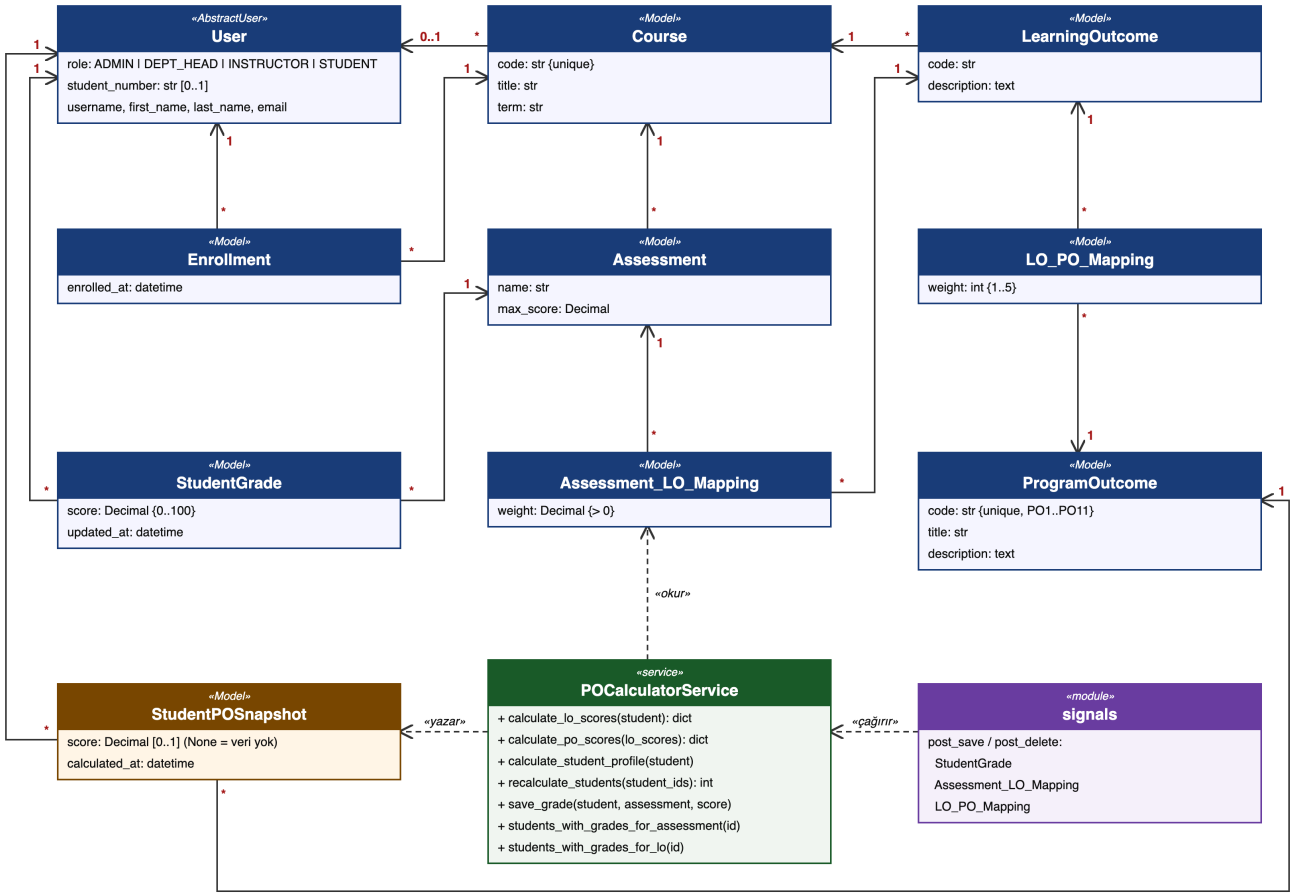
Sınıf	Sorumluluğu
`User`, `Course`, `Enrollment`, `LearningOutcome`, `ProgramOutcome`	Kurumsal yapı: kim, hangi ders, hangi çıktı
`Assessment`, `StudentGrade`	Değerlendirmeler ve ham notlar
`Assessment_LO_Mapping`, `LO_PO_Mapping`	Ağırlıklar: değerlendirme → LO (katsayı), LO → PO (1–5)
`StudentPOSnapshot`	Ön hesaplanmış PO profili; yalnızca servis yazar
`POCalculatorService`	Not → LO → PO hesabı, snapshot yazımı, etkilenen öğrencilerin bulunması
`signals`	Not, eşleme ya da ağırlık değişince servisi çağırır

Formül tek bir fonksiyondadır (`weighted_average`) ve iki adımda da o kullanılır; toplam ağırlık 0 ise 0 değil `None` (veri yok) döner. Örnek: LO1 = (0.60 × 90 + 0.40 × 80) ÷ 1.00 = 86.0; LO2 = (0.40 × 90 + 1.00 × 80) ÷ 1.40 = 82.9; PO1 = (2 × 86.0 + 1 × 82.86) ÷ 3 = 85.0; PO2 = 82.9. Hocanın 1.0 : 0.5 : 1.0 ağırlıkları modelde 1–5 tam sayı olduğu için 2 : 1 : 2 olarak girilir; sonuç aynıdır.

Waffle 5.1 — Sistem Mimarisi



Şekil 1 — Sistem mimarisi



Düz ok: ilişki (ForeignKey), * → 1. Kesikli ok: bağımlılık. signals, kayıt değişince servisi çağırır; servis kaynak tabloları okur, snapshot'ı yazar.

Şekil 2 — Class diyagramı